NAME

ovn-controller – Open Virtual Network local controller

SYNOPSIS

ovn−controller [*options*] [*ovs-database*]

DESCRIPTION

ovn−controller is the local controller daemon for OVN, the Open Virtual Network. It connects up to the OVN Southbound database (see **ovn−sb**(5)) over the OVSDB protocol, and down to the Open vSwitch database (see **ovs−vswitchd.conf.db**(5)) over the OVSDB protocol and to **ovs−vswitchd**(8) via OpenFlow. Each hypervisor and software gateway in an OVN deployment runs its own independent copy of **ovn−controller**; thus, **ovn−controller**'s downward connections are machine-local and do not run over a physical network.

ACL LOGGING

ACL log messages are logged through **ovn−controller**'s logging mechanism. ACL log entries have the module **acl_log** at log level **info**. Configuring logging is described below in the **Logging Options** section.

OPTIONS

Daemon Options

−−pidfile[**=***pidfile*]

Causes a file (by default, *program***.pid**) to be created indicating the PID of the running process. If the *pidfile* argument is not specified, or if it does not begin with **/**, then it is created in .

If **−−pidfile** is not specified, no pidfile is created.

−−overwrite−pidfile

By default, when **−−pidfile** is specified and the specified pidfile already exists and is locked by a running process, the daemon refuses to start. Specify **−−overwrite−pidfile** to cause it to instead overwrite the pidfile.

When **−−pidfile** is not specified, this option has no effect.

−−detach

Runs this program as a background process. The process forks, and in the child it starts a new session, closes the standard file descriptors (which has the side effect of disabling logging to the console), and changes its current directory to the root (unless **−−no−chdir** is specified). After the child completes its initialization, the parent exits.

−−monitor

Creates an additional process to monitor this program. If it dies due to a signal that indicates a programming error (**SIGABRT**, **SIGALRM**, **SIGBUS**, **SIGFPE**, **SIGILL**, **SIGPIPE**, **SIGSEGV**, **SIGXCPU**, or **SIGXFSZ**) then the monitor process starts a new copy of it. If the daemon dies or exits for another reason, the monitor process exits.

This option is normally used with **−−detach**, but it also functions without it.

−−no−chdir

By default, when **−−detach** is specified, the daemon changes its current working directory to the root directory after it detaches. Otherwise, invoking the daemon from a carelessly chosen directory would prevent the administrator from unmounting the file system that holds that directory.

Specifying **−−no−chdir** suppresses this behavior, preventing the daemon from changing its current working directory. This may be useful for collecting core files, since it is common behavior to write core dumps into the current working directory and the root directory is not a good directory to use.

This option has no effect when **−−detach** is not specified.

−−no−self−confinement

By default this daemon will try to self-confine itself to work with files under well-known directories determined at build time. It is better to stick with this default behavior and not to use this flag unless some other Access Control is used to confine daemon. Note that in contrast to other access

control implementations that are typically enforced from kernel-space (e.g. DAC or MAC), self-confinement is imposed from the user-space daemon itself and hence should not be considered as a full confinement strategy, but instead should be viewed as an additional layer of security.

−−user=*user***:***group*
Causes this program to run as a different user specified in *user***:***group*, thus dropping most of the root privileges. Short forms *user* and **:***group* are also allowed, with current user or group assumed, respectively. Only daemons started by the root user accepts this argument.

On Linux, daemons will be granted **CAP_IPC_LOCK** and **CAP_NET_BIND_SERVICES** before dropping root privileges. Daemons that interact with a datapath, such as **ovs−vswitchd**, will be granted three additional capabilities, namely **CAP_NET_ADMIN**, **CAP_NET_BROADCAST** and **CAP_NET_RAW**. The capability change will apply even if the new user is root.

Logging Options

−v[*spec*]
−−verbose=[*spec*]
Sets logging levels. Without any *spec*, sets the log level for every module and destination to **dbg**. Otherwise, *spec* is a list of words separated by spaces or commas or colons, up to one from each category below:

- A valid module name, as displayed by the **vlog/list** command on **ovs−appctl**(8), limits the log level change to the specified module.
- **syslog**, **console**, or **file**, to limit the log level change to only to the system log, to the console, or to a file, respectively. (If **−−detach** is specified, the daemon closes its standard file descriptors, so logging to the console will have no effect.)
- **off**, **emer**, **err**, **warn**, **info**, or **dbg**, to control the log level. Messages of the given severity or higher will be logged, and messages of lower severity will be filtered out. **off** filters out all messages. See **ovs−appctl**(8) for a definition of each log level.

Case is not significant within *spec*.

Regardless of the log levels set for **file**, logging to a file will not take place unless **−−log−file** is also specified (see below).

For compatibility with older versions of OVS, **any** is accepted as a word but has no effect.

−v
−−verbose
Sets the maximum logging verbosity level, equivalent to **−−verbose=dbg**.

−vPATTERN:*destination***:***pattern*
−−verbose=PATTERN:*destination***:***pattern*
Sets the log pattern for *destination* to *pattern*. Refer to **ovs−appctl**(8) for a description of the valid syntax for *pattern*.

−vFACILITY:*facility*
−−verbose=FACILITY:*facility*
Sets the RFC5424 facility of the log message. *facility* can be one of **kern**, **user**, **mail**, **daemon**, **auth**, **syslog**, **lpr**, **news**, **uucp**, **clock**, **ftp**, **ntp**, **audit**, **alert**, **clock2**, **local0**, **local1**, **local2**, **local3**, **local4**, **local5**, **local6** or **local7**. If this option is not specified, **daemon** is used as the default for the local system syslog and **local0** is used while sending a message to the target provided via the **−−syslog−target** option.

−−log−file[**=***file*]
Enables logging to a file. If *file* is specified, then it is used as the exact name for the log file. The default log file name used if *file* is omitted is **/usr/local/var/log/ovn/***program***.log**.

−−syslog−target=*host***:***port*
Send syslog messages to UDP *port* on *host*, in addition to the system syslog. The *host* must be a numerical IP address, not a hostname.

−−syslog−method=*method*

Specify *method* as how syslog messages should be sent to syslog daemon. The following forms are supported:

- **libc**, to use the libc **syslog()** function. Downside of using this options is that libc adds fixed prefix to every message before it is actually sent to the syslog daemon over **/dev/log** UNIX domain socket.
- **unix:***file*, to use a UNIX domain socket directly. It is possible to specify arbitrary message format with this option. However, **rsyslogd 8.9** and older versions use hard coded parser function anyway that limits UNIX domain socket use. If you want to use arbitrary message format with older **rsyslogd** versions, then use UDP socket to localhost IP address instead.
- **udp:***ip***:***port*, to use a UDP socket. With this method it is possible to use arbitrary message format also with older **rsyslogd**. When sending syslog messages over UDP socket extra precaution needs to be taken into account, for example, syslog daemon needs to be configured to listen on the specified UDP port, accidental iptables rules could be interfering with local syslog traffic and there are some security considerations that apply to UDP sockets, but do not apply to UNIX domain sockets.
- **null**, to discard all messages logged to syslog.

The default is taken from the **OVS_SYSLOG_METHOD** environment variable; if it is unset, the default is **libc**.

PKI Options

PKI configuration is required in order to use SSL/TLS for the connections to the Northbound and Southbound databases.

−p *privkey.pem*
−−private−key=*privkey.pem*

Specifies a PEM file containing the private key used as identity for outgoing SSL/TLS connections.

−c *cert.pem*
−−certificate=*cert.pem*

Specifies a PEM file containing a certificate that certifies the private key specified on **−p** or **−−private−key** to be trustworthy. The certificate must be signed by the certificate authority (CA) that the peer in SSL/TLS connections will use to verify it.

−C *cacert.pem*
−−ca−cert=*cacert.pem*

Specifies a PEM file containing the CA certificate for verifying certificates presented to this program by SSL/TLS peers. (This may be the same certificate that SSL/TLS peers use to verify the certificate specified on **−c** or **−−certificate**, or it may be a different one, depending on the PKI design in use.)

−C none
−−ca−cert=none

Disables verification of certificates presented by SSL/TLS peers. This introduces a security risk, because it means that certificates cannot be verified to be those of known trusted hosts.

−−ssl−server−name=*servername*

Specifies the server name to use for TLS Server Name Indication (SNI). By default, the hostname from the connection string is used for SNI. This option allows overriding the SNI hostname, which is useful when connecting through proxies or service meshes where the connection endpoint differs from the intended server name.

−−bootstrap−ca−cert=*cacert.pem*

When *cacert.pem* exists, this option has the same effect as **−C** or **−−ca−cert**. If it does not exist, then the executable will attempt to obtain the CA certificate from the SSL/TLS peer on its first SSL/TLS connection and save it to the named PEM file. If it is successful, it will immediately drop the connection and reconnect, and from then on all SSL/TLS connections must be authenticated by a certificate signed by the CA certificate thus obtained.

This option exposes the SSL/TLS connection to a man-in-the-middle attack obtaining the initial CA certificate, but it may be useful for bootstrapping.

This option is only useful if the SSL/TLS peer sends its CA certificate as part of the SSL/TLS certificate chain. SSL/TLS protocols do not require the server to send the CA certificate.

This option is mutually exclusive with **−C** and **−−ca−cert**.

−−peer−ca−cert=*peer-cacert.pem*

Specifies a PEM file that contains one or more additional certificates to send to SSL/TLS peers. *peer-cacert.pem* should be the CA certificate used to sign the program's own certificate, that is, the certificate specified on **−c** or **−−certificate**. If the program's certificate is self-signed, then **−−certificate** and **−−peer−ca−cert** should specify the same file.

This option is not useful in normal operation, because the SSL/TLS peer must already have the CA certificate for the peer to have any confidence in the program's identity. However, this offers a way for a new installation to bootstrap the CA certificate on its first SSL/TLS connection.

Other Options

−−unixctl=*socket*

Sets the name of the control socket on which *program* listens for runtime management commands (see *RUNTIME MANAGEMENT COMMANDS,* below). If *socket* does not begin with **/**, it is interpreted as relative to . If **−−unixctl** is not used at all, the default socket is **/***program***.***pid***.ctl**, where *pid* is *program*'s process ID.

Specifying **none** for *socket* disables the control socket feature.

−n *chassis*
−−chassis=*chassis*

Use *chassis* as the chassis name. This takes precedence over the name in the **system−id−override** file and **external_ids:system−id** in the local Open vSwitch database.

−−dump−inc−proc−graph[**=***node*]

Dump the incremental processing engine graph in DOT format to stdout and then exit. If *node* is specified, only the subgraph rooted at that engine node is printed.

−h
−−help Prints a brief help message to the console.

−V
−−version

Prints version information to the console.

CONFIGURATION

ovn−controller retrieves most of its configuration information from the local Open vSwitch's ovsdb-server instance. The default location is **db.sock** in the local Open vSwitch's "run" directory. It may be overridden by specifying the *ovs-database* argument as an OVSDB active or passive connection method, as described in **ovsdb**(7).

ovn−controller assumes it gets configuration information from the following keys in the **Open_vSwitch** table of the local OVS instance:

external_ids:system−id
The chassis name to use in the Chassis table. Changing the **system−id** while **ovn−controller** is running is not directly supported. Users have two options: either first gracefully stop **ovn−controller** or manually delete the stale **Chassis** and **Chassis_Private** records after changing the **system−id**. Note that the chassis name can also be provided via the **system−id−override** file in the local OVN "etc" directory or via the **−n** command-line option. The following precedence is used: first, the command-line option is read; if not present, the **system−id−override** file is read; if not present, then the name configured in the database is used.

external_ids:hostname
The hostname to use in the Chassis table.

external_ids:ovn−bridge
The integration bridge to which logical ports are attached. The default is **br−int**. If this bridge does not exist when ovn-controller starts, it will be created automatically with the default configuration suggested in **ovn−architecture**(7). When more than one controllers are running on the same host, **external_ids:ovn−bridge−CHASSIS_NAME** should be set for each of them, pointing to a unique bridge. This is required to avoid controllers stepping on each others' feet.

external_ids:ovn−bridge−datapath−type
This configuration is optional. If set, then the datapath type of the integration bridge will be set to the configured value. If this option is not set, then **ovn−controller** will not modify the existing **datapath−type** of the integration bridge.

external_ids:ovn−remote
The OVN database that this system should connect to for its configuration, in one of the same forms documented above for the *ovs-database*.

external_ids:ovn−monitor−all
A boolean value that tells if **ovn−controller** should monitor all records of tables in the **OVN_Southbound**. If this option is set to **false**, ovn-controller will conditionally monitor only the records that are needed for the local chassis.

It is more efficient to set it to **true** in use cases where the chassis would anyway need to monitor most of the records in the **OVN_Southbound** database, which would save the overhead of conditions processing, especially for server side. Typically, set it to **true** for environments where all workloads need to be reachable from each other.

NOTE: for efficiency and scalability in common scenarios **ovn−controller** unconditionally monitors all sub-ports (ports with **parent_port** set) regardless of the **ovn−monitor−all** value.

Default value is *false*.

external_ids:ovn−remote−probe−interval
The inactivity probe interval of the connection to the OVN database, in milliseconds. If the value is zero, it disables the connection keepalive feature.

If the value is nonzero, then it will be forced to a value of at least 1000 ms.

external_ids:ovn−encap−type
The encapsulation type that a chassis should use to connect to this node. Multiple encapsulation types may be specified with a comma-separated list. Each listed encapsulation type will be paired with **ovn−encap−ip**.

Supported tunnel types for connecting hypervisors and gateways are **geneve** and **vxlan**.

Due to the limited amount of metadata in **vxlan**, the capabilities and performance of connected gateways and hypervisors will be reduced versus other tunnel formats.

external_ids:ovn−encap−ip
The IP address that a chassis should use to connect to this node using encapsulation types specified by **external_ids:ovn−encap−type**. Multiple encapsulation IPs may be specified with a comma-separated list.

In scenarios where multiple encapsulation IPs are present, distinct tunnels are established for each remote chassis. These tunnels are differentiated by setting unique **options:local_ip** and **options:remote_ip** values in the tunnel interface. When transmitting a packet to a remote chassis, the selection of local_ip is guided by the **Interface:external_ids:encap−ip** from the local OVSDB, corresponding to the VIF originating the packet, if specified. The **Interface:external_ids:encap−ip** setting of the VIF is also populated to the **Port_Binding** table in the OVN SB database via the **encap** column. Consequently, when a remote chassis needs to send a packet to a port-binding associated with this VIF, it utilizes the tunnel with the appropriate **options:remote_ip** that matches the **ip** in **Port_Binding:encap**. This mechanism is particularly beneficial for chassis with multiple physical interfaces designated for tunneling, where each interface is optimized for handling specific traffic associated with particular VIFs.

external_ids:ovn−encap−ip−default
When **ovn−encap−ip** contains multiple IPs, this field indicates the default one.

external_ids:ovn−encap−df_default
indicates the DF flag handling of the encapulation. Set to **true** to set the DF flag for new data paths or **false** to clear the DF flag.

external_ids:ovn−enable−flow−based−tunnels
The boolean flag indicates if **ovn−controller** should use flow-based tunnels instead of port-based tunnels for overlay network connectivity. Default value is **false**.

Port−based tunnels (default mode) create a dedicated tunnel port for each combination of remote chassis and local encap IP, while **Flow−based tunnels** create a single shared tunnel port for each tunnel type, and use OpenFlow **set_field** actions to dynamically set tunnel source and destination IP addresses per packet.

This feature is experimental and is disabled by default. There are some known limitations to the feature:

- IPSec is not supported.
- BFD between tunnel endpoints is not supported, thus HA chassis (e.g. for Distributed Gateway Port redundancy) is not supported.

external_ids:ovn−bridge−mappings
A list of key-value pairs that map a physical network name to a local ovs bridge that provides connectivity to that network. An example value mapping two physical network names to two ovs bridges would be: **physnet1:br−eth0,physnet2:br−eth1**.

external_ids:ovn−encap−csum
ovn−encap−csum indicates that encapsulation checksums can be transmitted and received with reasonable performance. It is a hint to senders transmitting data to this chassis that they should use checksums to protect OVN metadata. Set to **true** to enable or **false** to disable. Depending on the capabilities of the network interface card, enabling encapsulation checksum may incur performance loss. In such cases, encapsulation checksums can be disabled.

external_ids:ovn−encap−tos
ovn−encap−tos indicates the value to be applied to OVN tunnel interface's option:tos as specified in the Open_vSwitch database Interface table. Please refer to Open VSwitch Manual for details.

external_ids:ovn−cms−options
A list of options that will be consumed by the CMS Plugin and which specific to this particular chassis. An example would be: **cms_option1,cms_option2:foo**.

external_ids:ovn−transport−zones
The transport zone(s) that this chassis belongs to. Transport zones is a way to group different chassis so that tunnels are only formed between members of the same group(s). Multiple transport zones may be specified with a comma-separated list. For example: tz1,tz2,tz3.

If not set, the Chassis will be considered part of a default transport zone.

external_ids:ovn−chassis−mac−mappings
A list of key-value pairs that map a chassis specific mac to a physical network name. An example value mapping two chassis macs to two physical network names would be: **physnet1:aa:bb:cc:dd:ee:ff,physnet2:a1:b2:c3:d4:e5:f6**. These are the macs that ovn-controller will replace a router port mac with, if packet is going from a distributed router port on vlan type logical switch.

external_ids:ovn−is−interconn
The boolean flag indicates if the chassis is used as an interconnection gateway.

external_ids:ovn−match−northd−version
The boolean flag indicates if **ovn−controller** needs to check **ovn−northd** version. If this flag is set to true and the **ovn−northd's** version (reported in the Southbound database) doesn't match with the **ovn−controller's** internal version, then it will stop processing the southbound and local Open vSwitch database changes. The default value is considered false if this option is not defined.

external_ids:ovn−enable−lflow−cache
The boolean flag indicates if **ovn−controller** should enable/disable the logical flow in-memory cache it uses when processing Southbound database logical flow changes. By default caching is enabled.

external_ids:ovn−limit−lflow−cache
When used, this configuration value determines the maximum number of logical flow cache entries **ovn−controller** may create when the logical flow cache is enabled. By default the size of the cache is unlimited.

external_ids:ovn−memlimit−lflow−cache−kb
When used, this configuration value determines the maximum size of the logical flow cache (in KB) **ovn−controller** may create when the logical flow cache is enabled. By default the size of the cache is unlimited.

external_ids:ovn−trim−limit−lflow−cache
When used, this configuration value sets the minimum number of entries in the logical flow cache starting with which automatic memory trimming is performed. By default this is set to 10000 entries.

external_ids:ovn−trim−wmark−perc−lflow−cache
When used, this configuration value sets the percentage from the high watermark number of entries in the logical flow cache under which automatic memory trimming is performed. E.g., if the trim watermark percentage is set to 50%, automatic memory trimming happens only when the number of entries in the logical flow cache gets reduced to less than half of the last measured high watermark. By default this is set to 50.

external_ids:ovn−trim−timeout−ms
When used, this configuration value specifies the time, in milliseconds, since the last logical flow cache operation after which **ovn−controller** performs memory trimming regardless of how many entries there are in the cache. By default this is set to 30000 (30 seconds).

external_ids:garp−max−timeout−sec
When used, this configuration value specifies the maximum timeout (in seconds) between two consecutive GARP packets sent by **ovn−controller**. **ovn−controller** by default sends just 4 GARP packets with an exponential backoff timeout. Setting **external_ids:garp−max−timeout−sec** allows to cap for the exponential backoff used by **ovn−controller** to send GARPs packets.

external_ids:arp−nd−max−timeout−sec
When used, this configuration value specifies the maximum timeout (in seconds) between two consecutive ARP/ND packets sent by **ovn−controller** to resolve ECMP nexthop mac address. **ovn−controller** by default continuously sends ARP/ND packets. Setting **external_ids:arp−nd−max−timeout−sec** allows to cap for the exponential backoff used by **ovn−controller**
to send ARPs/NDs packets.

external_ids:ovn−bridge−remote
Connection to the OVN management bridge in OvS. It defaults to **unix:***br-int***.mgmt** when not specified.

external_ids:ovn−bridge−remote−probe−interval
The inactivity probe interval of the connection to the OVN management bridge, in milliseconds. It defaults to zero. If the value is zero, it disables the inactivity probe.

external_ids:dynamic−routing−port−mapping
This setting works together with the Northbound dynamic-routing-port-name option on Logical_Router_Ports. See the **ovn−nb**(5) for more details.

Note: When using the **routing−protocol−redirect** option with veth pairs on Linux systems, this mapping may not be necessary as **ovn−controller** can automatically discover the veth peer interface name. See the **routing−protocol−redirect** option documentation in **ovn−nb**(5) for details.

external_ids:ovn−cleanup−on−exit
A boolean value that controls whether **ovn−controller** releases this chassis's logical port bindings and removes its **Chassis**, **Chassis_Private**, **Encap**, and **IGMP_Group** rows when it terminates through the **exit** runtime command. The default value is **true**.

If set to **false**, cleanup is skipped as if **exit −−restart** had been used. The **−−restart** argument always skips cleanup, regardless of this setting. The chassis-specific form, **external_ids:ovn−cleanup−on−exit−[chassis]**, is also supported as described below.

external_ids:ovn−evpn−vxlan−ports
Comma separated list of UDP ports used as a destination port for the EVPN tunnels created by this ovn-controller.

external_ids:ovn−evpn−local−ip
IP address list used as a source addresses for the EVPN traffic leaving this OVN setup. The **ovn−evpn−local−ip** can be specified by the CMS according to the following syntax: **external_ids:ovn−evpn−local−ip=vni0−IPv4,vni1−IPv4, vni1−IPv6,IPv4,IPv6**, where if no VNI value is specified, OVN will use the provided IP as default IP.

Most of configuration options listed above can also be set for a particular chassis name (see **external_ids:system−id** for more information). This can be achieved by setting **external_ids:option−[chassis]** instead of **external_ids:option**. For example, set **external_ids:ovn−encap−ip−otherhv** to use a particular IP address for the controller instance named **otherhv**. Name specific configuration options always override any global options set in the database.

Chassis-specific configuration options in the database plus the ability to configure the chassis name to use via the **system−id−override** file or command line allows to run multiple **ovn−controller** instances with unique chassis names on the same host using the same **vswitchd** instance. This may be useful when running a hybrid setup with more than one CMS managing ports on the host, or to use different datapath types

on the same host. In order to support stateful features (ACLs, LBs, NAT) in multiple OVN deployments co-hosted on the same chassis, CMS also need to separate conntrack zone ranges between the co-hosted ovn-controllers through the **external_ids:ct−zone−range** configuration described in upcoming sections.

ovn−controller reads the following values from the **Open_vSwitch** database of the local OVS instance:

datapath−type from **Bridge** table
: This value is read from local OVS integration bridge row of **Bridge** table and populated in **other_config:datapath-type** of the **Chassis** table in the OVN_Southbound database.

iface−types from **Open_vSwitch** table
: This value is populated in **other_config:iface-types** of the **Chassis** table in the OVN_Southbound database.

private_key, **certificate**, **ca_cert**, and **bootstrap_ca_cert** from **SSL** table
: These values provide the SSL/TLS configuration used for connecting to the OVN southbound database server when an SSL/TLS connection type is configured via **external_ids:ovn−remote**. Note that this SSL/TLS configuration can also be provided via command-line options, the configuration in the database takes precedence if both are present.

OPEN VSWITCH DATABASE USAGE

ovn−controller uses a number of **external_ids** keys in the Open vSwitch database to keep track of ports and interfaces. For proper operation, users should not change or clear these keys:

external_ids:ovn−chassis−id in the **Port** table
: The presence of this key identifies a tunnel port within the integration bridge as one created by **ovn−controller** to reach a remote chassis. Its value is the chassis ID of the remote chassis.

external_ids:ct−zone−range in the **Open_vSwitch** table
: The presence of this key identifies a minimum and maximum values for ct-zone ids dynamically selected by ovn-controller (boundaries are included in the range). Minimum value is 1 while maximum value is 65535.

external_ids:ct−zone−* in the **Bridge** table
: Logical ports and gateway routers are assigned a connection tracking zone by **ovn−controller** for stateful services. To keep state across restarts of **ovn−controller**, these keys are stored in the integration bridge's Bridge table. The name contains a prefix of **ct−zone−** followed by the name of the logical port or gateway router's zone key. The value for this key identifies the zone used for this port.

external_ids:ovn−localnet−port in the **Port** table
: The presence of this key identifies a patch port as one created by **ovn−controller** to connect the integration bridge and another bridge to implement a **localnet** logical port. Its value is the name of the logical port with **type** set to **localnet** that the port implements. See **external_ids:ovn−bridge−mappings**, above, for more information.

  Each **localnet** logical port is implemented as a pair of patch ports, one in the integration bridge, one in a different bridge, with the same **external_ids:ovn−localnet−port** value.

external_ids:ovn−l2gateway−port in the **Port** table
: The presence of this key identifies a patch port as one created by **ovn−controller** to connect the integration bridge and another bridge to implement a **l2gateway** logical port. Its value is the name of the logical port with **type** set to **l2gateway** that the port implements. See **external_ids:ovn−bridge−mappings**, above, for more information.

  Each **l2gateway** logical port is implemented as a pair of patch ports, one in the integration bridge, one in a different bridge, with the same **external_ids:ovn−l2gateway−port** value.

external_ids:ovn−l3gateway−port in the **Port** table
: Older versions of **ovn−controller** used this key on patch ports. Current versions do not create or use such ports. During normal operation after the startup delay, they remove obsolete patch-port pairs discovered through a port with this key on the integration bridge.

external_ids:ovn−logical−patch−port in the **Port** table
: Older versions of **ovn−controller** used this key on patch ports. Current versions do not create or use such ports. During normal operation after the startup delay, they remove obsolete patch-port pairs discovered through a port with this key on the integration bridge.

external_ids:ovn−startup−ts in the **Bridge** table
: This key represents the timestamp (in milliseconds) at which **ovn−controller** process was started.

external_ids:ovn−nb−cfg in the **Bridge** table
: This key represents the last known **OVN_Southbound.SB_Global.nb_cfg** value for which all flows have been successfully installed in OVS.

external_ids:ovn−nb−cfg−ts in the **Bridge** table
: This key represents the timestamp (in milliseconds) of the last known **OVN_Southbound.SB_Global.nb_cfg** value for which all flows have been successfully installed in OVS.

external_ids:ovn−nb−cfg−sb−ts in the **Bridge** table
: This key records the value of **OVN_Southbound.SB_Global.nb_cfg_timestamp** that corresponded to the **ovn−nb−cfg** generation for which all flows have been successfully installed in OVS. It is the wall-clock time (in milliseconds since the Unix epoch) at which **ovn−northd** wrote that **nb_cfg** value to the Southbound database. The difference between this timestamp and **ovn−nb−cfg−ts** gives the per-chassis end-to-end propagation latency for that configuration generation. The key is absent when no timestamp is available (e.g. when **NB_Global.options:enable_chassis_nb_cfg_update** is **false**).

external_ids:ovn−installed and **external_ids:ovn−installed−ts** in the **Interface** table
: This key is set after all openflow operations corresponding to the OVS interface have been processed by ovs-vswitchd. At the same time a timestamp, in milliseconds since the epoch, is stored in **external_ids:ovn−installed−ts**.

external_ids:ovn−evpn−tunnel in the **Port** and **Interface** table
: Presence of this key indicates that the **Port/Interface** was created by ovn-controller to be used as tunnel port for EVPN traffic.

OVN SOUTHBOUND DATABASE USAGE

ovn−controller reads from much of the **OVN_Southbound** database to guide its operation. **ovn−controller** also writes to the following tables:

Chassis
: **ovn−controller** creates and maintains the row that represents its chassis. It writes the **name**, **hostname**, **encaps**, **other_config**, and **transport_zones** columns. It removes the row during exit cleanup.

Chassis_Private
: **ovn−controller** creates and maintains the private row for its chassis. It writes the **name**, **chassis**, **nb_cfg**, and **nb_cfg_timestamp** columns. It removes the row during exit cleanup.

Encap **ovn−controller** creates and maintains rows for the tunnel encapsulations by which its chassis can be reached. It writes the **type**, **ip**, **options**, and **chassis_name** columns. These non-root rows are removed with their owning **Chassis** row during exit cleanup.

Port_Binding
: **ovn−controller** claims and releases logical ports by updating the **chassis** and **additional_chassis** columns. It also updates the **up**, **encap**, **additional_encap**,

virtual_parent, and **options** columns when required by the corresponding port behavior. Exit cleanup releases bindings owned by this chassis.

MAC_Binding
: **ovn−controller** creates and updates rows as instructed by **put_arp** and **put_nd** logical actions, while simulating GARPs on peer router datapaths, and when synchronizing EVPN neighbors. It also refreshes timestamps for active bindings. It writes the **logical_port**, **ip**, **mac**, **datapath**, and **timestamp** columns. These changes persist beyond the lifetime of **ovn−controller**, except that EVPN-synchronized rows are removed when their corresponding neighbors become stale.

Controller_Event
: **ovn−controller** inserts events generated by controller actions. It writes the **event_type**, **event_info**, **chassis**, and **seq_num** columns.

IGMP_Group
: **ovn−controller** creates, updates, and removes rows for multicast groups learned on its chassis. It writes the **address**, **protocol**, **datapath**, **chassis**, **ports**, and **chassis_name** columns. It removes rows owned by this chassis during exit cleanup.

Service_Monitor
: **ovn−controller** writes the **status** column with the result of each service health check. See **ovn−sb**(5) for the status values.

BFD
: **ovn−controller** writes the **status** column as BFD sessions change state. See **ovn−sb**(5) for the state values.

FDB
: **ovn−controller** creates and updates forwarding database entries as instructed by the **put_fdb** logical action. It also refreshes timestamps for active entries. It writes the **mac**, **dp_key**, **port_key**, and **timestamp** columns.

Learned_Route
: When built with Linux Netlink support (**HAVE_NETLINK**), **ovn−controller** synchronizes routes learned from the host routing table. It creates and removes rows and writes the **datapath**, **logical_port**, **ip_prefix**, and **nexthop** columns.

RUNTIME MANAGEMENT COMMANDS

ovn−appctl can send commands to a running **ovn−controller** process. The following list describes commands specific to **ovn−controller** and its incremental processing engine. Shared OVS libraries can register additional commands; use **ovn−appctl −t ovn−controller list−commands** to list the commands available in a running process.

exit [**−−restart**]
: Causes **ovn−controller** to gracefully terminate. With **−−restart**, it skips the database cleanup normally done on exit.

ct−zone−list
: Lists each local logical port and its connection tracking zone.

evpn/remote−vtep−list
: Lists the remote EVPN VTEPs learned from the host neighbor table, including their IP addresses, UDP ports, and VNIs. This command is registered in all builds, but when **ovn−controller** is built without Linux Netlink support (**HAVE_NETLINK**), it does not send a reply and **ovn−appctl** waits indefinitely by default.

evpn/vtep−arp−list
: Lists the EVPN ARP and IPv6 neighbor entries maintained by **ovn−controller**.

evpn/vtep−binding−list
: Lists the bindings between remote EVPN VTEPs and OVN datapaths.

evpn/vtep−fdb−list
Lists the EVPN forwarding database entries maintained by **ovn−controller**.

evpn/vtep−multicast−group−list
Lists the EVPN multicast groups and their remote VTEP bindings.

meter−table−list
Lists each meter table entry and its local meter id.

group−table−list
Lists each group table entry and its local group id.

inject−pkt *microflow*
Injects *microflow* into the connected Open vSwitch instance. *microflow* must contain an ingress logical port (**inport** argument) that is present on the Open vSwitch instance.

The *microflow* argument describes the packet whose forwarding is to be simulated, in the syntax of an OVN logical expression, as described in **ovn−sb**(5), to express constraints. The parser understands prerequisites; for example, if the expression refers to **ip4.src**, there is no need to explicitly state **ip4** or **eth.type == 0x800**.

connection−status
Show OVN SBDB connection status for the chassis.

recompute
Trigger a full compute iteration in **ovn−controller** based on the contents of the Southbound database and local OVS database. This command is deprecated; use **inc−engine/recompute** instead.

This command is intended to use only in the event of a bug in the incremental processing engine in **ovn−controller** to avoid inconsistent states. It should therefore be used with care as full recomputes are cpu intensive.

sb−cluster−state−reset
Reset southbound database cluster status when databases are destroyed and rebuilt.

If all databases in a clustered southbound database are removed from disk, then the stored index of all databases will be reset to zero. This will cause ovn-controller to be unable to read or write to the southbound database, because it will always detect the data as stale. In such a case, run this command so that ovn-controller will reset its local index so that it can interact with the southbound database again.

debug/delay−nb−cfg−report *seconds*
This command is used to delay ovn-controller updating the **nb_cfg** back to **OVN_Southbound** database. This is useful when **ovn−nbctl −−wait=hv** is used to measure end-to-end latency in a large scale environment. See **ovn−nbctl**(8) for more details.

debug/dump−lflow−conj−ids
Dumps the logical flow conjunction IDs currently allocated by **ovn−controller**.

debug/dump−local−bindings
Dumps the local logical port binding data maintained by **ovn−controller**.

debug/dump−local−datapaths
Dumps the local datapaths maintained by **ovn−controller**.

debug/dump−local−template−vars
Dumps the chassis-local template variables maintained by **ovn−controller**.

debug/dump−mac−bindings
Dumps the local MAC bindings maintained by **ovn−controller**.

debug/dump−peer−ports
Dumps the peer ports associated with local datapaths.

debug/dump−related−ports
Dumps the logical ports related to this chassis.

debug/enable−timewarp
Enables the time-warp facility and registers the **time/stop** and **time/warp** commands, which respectively stop and advance the monotonic clock for debugging and testing time-based behavior.

debug/pause
Pauses the main **ovn−controller** processing loop for debugging.

debug/resume
Resumes a processing loop paused by **debug/pause**.

debug/status
Reports whether the main processing loop is running or paused.

lflow−cache/flush
Flushes the **ovn−controller** logical flow cache.

lflow−cache/show−stats
Displays logical flow cache statistics: enabled/disabled, per cache type entry counts.

flush−lflow−cache
Flushes the logical flow cache. This command is deprecated; use **lflow−cache/flush** instead.

inc−engine/show−stats
Displays the following **ovn−controller** counters for each engine node:

- **recompute**
- **compute**
- **cancel**

inc−engine/show−stats *engine_node_name* [*counter_name*]
Display the **ovn−controller** engine counter(s) for the specified *engine_node_name*. *counter_name* is optional and can be one of **recompute**, **compute** or **cancel**.

inc−engine/clear−stats
Reset **ovn−controller** engine counters.

inc−engine/recompute
Triggers a full recompute of the incremental processing engine on the next iteration.

inc−engine/compute−log−timeout *msecs*
Sets the threshold in milliseconds that determines the log severity for engine recomputes and incremental change handlers. Events taking more than *msecs* milliseconds are logged at **INFO**, subject to rate limiting; shorter events and events taking exactly *msecs* milliseconds are logged at **DEBUG**. The default threshold is 500 milliseconds.

inc−engine/list−stopwatches [*node*]
Lists the engine nodes and their change handler names. If *node* is specified, only the handlers for that engine node are listed.

inc−engine/enable−stopwatch [*node*]
Enables the per-node and per-handler stopwatches in the incremental processing engine. If *node* is specified, only that node's stopwatches are enabled; otherwise all nodes are affected. While enabled, every engine node recompute and change handler invocation is timed and the results can be viewed with the **stopwatch/show** command. Stopwatches are disabled by default because they add measurable CPU overhead.

inc−engine/disable−stopwatch [*node*]
: Disables the per-node and per-handler stopwatches in the incremental processing engine. If *node* is specified, only that node's stopwatches are disabled; otherwise all nodes are affected. This is the default state.